

MODULE 10.1

Federated Learning Fundamentals

Privacy-Preserving Distributed Machine Learning

Edge devices
Inference
Can we use (i.e.) for training our models?

Overview



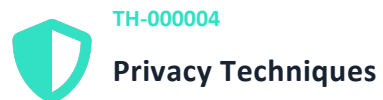
What it is, why it matters



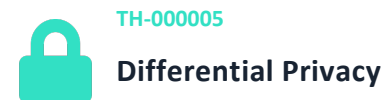
The canonical aggregation algorithm



Horizontal, vertical, transfer



DP, secure aggregation, HE



ϵ , δ , noise mechanisms



Flower framework · 30 min lab

LEARNING OBJECTIVES

Define

Explain what federated learning is and when to use it over centralised training

Derive

Step through the FedAvg algorithm and understand the aggregation math

Distinguish

Differentiate horizontal, vertical, and transfer federated learning

Apply

Select appropriate privacy mechanisms (DP, secure aggregation, HE) for a use case

Quantify

Reason about privacy budgets ϵ and δ and their trade-offs with accuracy

Implement

Build a minimal federated learning loop using the Flower framework

Why Federated Learning?

The problem with centralising sensitive data

Federated Learning (FL) is a machine learning paradigm in which a model is trained across multiple decentralised devices or silos **without raw data ever leaving its source.**

- Data stays on device — only model updates (gradients/weights) travel
- Enables collaboration without sharing competitive or sensitive information
- Regulatory compliance built-in: GDPR, HIPAA, CCPA
- Devices improve a shared model while maintaining local privacy

80%

of healthcare data never leaves the hospital that generated it

GDPR / HIPAA

prohibit raw data transfer across org boundaries

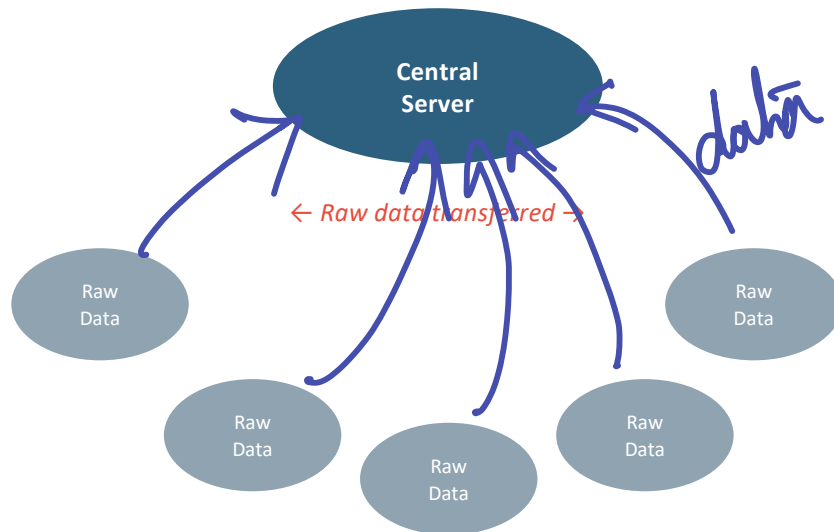
3-10×

less bandwidth than sending raw data to a central server

Centralised vs. Federated Training

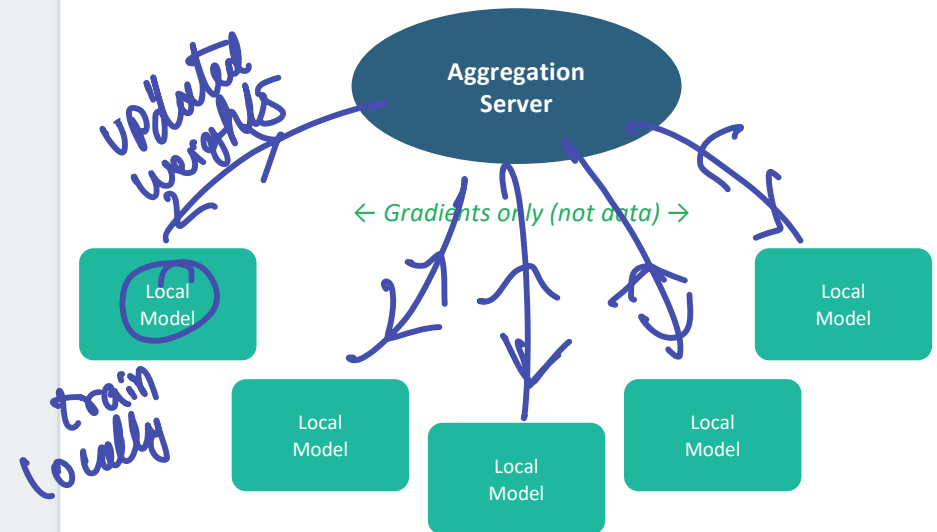
The fundamental architectural shift

Centralised Training



- ✗ **Privacy risk:** Data leaves source
- ✗ **Bandwidth:** All raw data uploaded
- ✗ **Compliance:** Violates GDPR/HIPAA

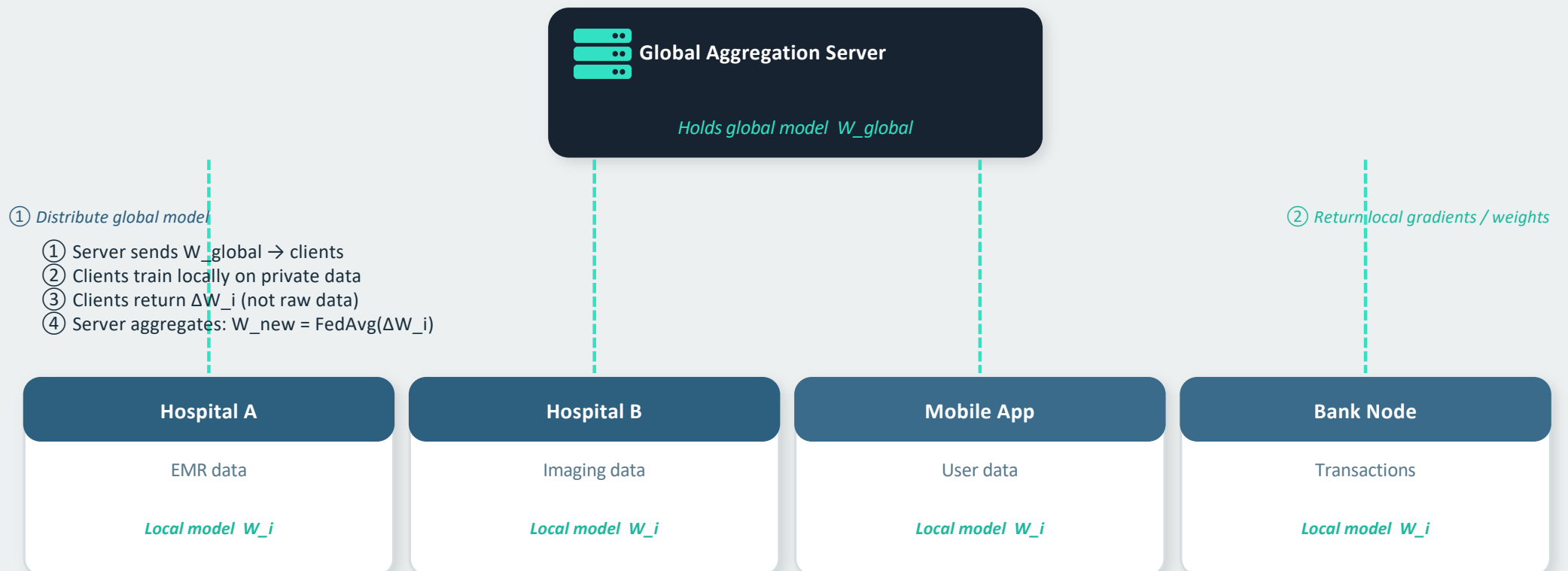
Federated Training



- ✓ **Privacy:** Data never leaves device
- ✓ **Bandwidth:** Only model updates sent
- ✓ **Compliance:** GDPR/HIPAA compatible

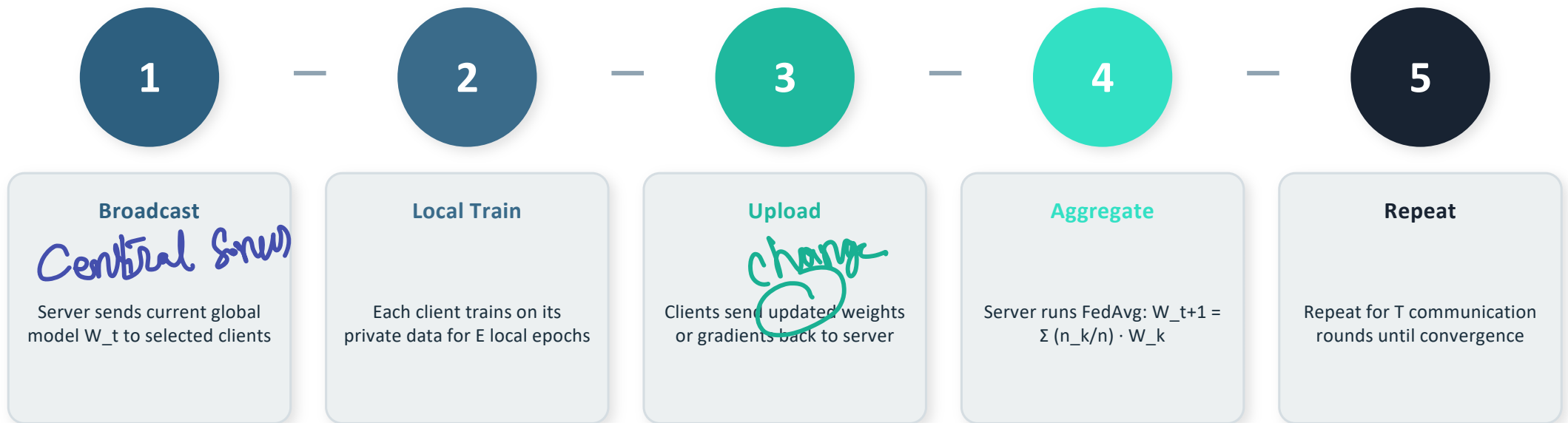
FL System Architecture

Server ↔ Clients ↔ Communication rounds



The Communication Round

One iteration of the federated training loop



Key Insight: Raw data never leaves the client. Only model parameters travel. The privacy guarantee comes from **what is NOT sent**.

Federated Averaging (FedAvg)

McMahan et al., 2017 — the canonical FL algorithm

Algorithm FedAvg (McMahan et al., 2017): Select a random subset of K clients each round. Each client runs stochastic gradient descent for E epochs on its local data. The server aggregates by taking a weighted average:

$$\mathbf{w}_{t+1} = \sum_k (n_k / n) \cdot \mathbf{w}_k$$
, where n_k is the local dataset size.

Symbol	Meaning	Typical Value
K	Number of clients selected per round	10–100
C	Fraction of clients selected (C = K/N)	0.1 (10%)
E	Local epochs per round	1–5
B	Local mini-batch size	32–256
η	Local learning rate	0.01–0.1
T	Total communication rounds	100–1000

FedAvg — Traced Example

N=4 clients, K=2 selected, Round t=1

Client 1 (Hospital)	Client 2 (Clinic)	Client 3 (Lab)	Client 4 (Pharmacy)
n_k = 500 samples	n_k = 200 samples	n_k = 800 samples	n_k = 300 samples
W_k: [0.3, 0.7, 0.2]	W_k: [0.1, 0.9, 0.4]	NOT selected this round	NOT selected this round
✓ SELECTED	✓ SELECTED		

③ Server Aggregates (FedAvg):

$$n = n_1 + n_2 = 500 + 200 = 700 \quad W_{\text{new}} = (500/700) \cdot [0.3, 0.7, 0.2] + (200/700) \cdot [0.1, 0.9, 0.4]$$

Result: $W_{\text{new}} \approx [0.243, 0.757, 0.257]$ — broadcast to all clients for round t=2

FedAvg — The Mathematics

Understanding the weighted aggregation and convergence properties

Objective

$$\min_W F(W) = \sum_k (n_k/n) \cdot F_k(W)$$

Global loss = weighted sum of local losses

Local Update (SGD)

$$W_k^{(t+1)} = W_k^{(t)} - \eta \cdot \nabla F_k(W_k^{(t)})$$

Each client runs E steps of local SGD

Global Aggregation

$$W^{(t+1)} = \sum_k (n_k/n) \cdot W_k^{(t+1)}$$

Server takes weighted average of client weights

Client Fraction

$$K = \max(C \cdot N, 1) \quad \text{clients sampled per round}$$

$C \in (0,1]$: controls communication frequency

 *Non-IID data (different distributions per client) causes client drift — addressed in slide 26*

§2

Types of Federated Learning

Horizontal · Vertical · Transfer · Cross-Silo vs Cross-Device

Horizontal Federated Learning

Same feature space, different data samples (most common)

Data Partition (row-wise — same features, different patients):

Age	BP	BMI	Glucose	Outcome

Hospital A (rows 1-500)

Hospital B (rows 501-900)

Hospital C (rows 901-1300)

Characteristics

- Same features
- Different samples
- Most common FL type
- Example: hospitals sharing patient records across different patient pools



Healthcare

Hospitals training on different patient cohorts with same EHR features



Mobile/IoT

Millions of devices training on next-word prediction (Google GBoard)



Finance

Banks training fraud models on different customer transaction histories

Vertical Federated Learning

Same users, different feature spaces — requires alignment

Data Partition (column-wise — same users, different features):

User ID	Age	Income	Credit	Purchases	Returns	Platform	Device	Time
← Bank (Party A) →				← Retailer (B) →		← Platform (C) →		

Key Challenge: Parties share the same users (overlapping entity set) but hold different features. Requires **private entity alignment (PSI)** to find common users before training, without revealing who those users are.

	Horizontal FL	Vertical FL
Users	Different users, same features	Same users, different features
Entity align?	No	Yes (PSI required)
Complexity	Lower	Higher (secure computation)
Example	Multi-hospital EHR	Bank + Retailer on shared customers

Federated Transfer Learning (FTL)

Different users AND different features — the most general case

What is FTL?

Federated Transfer Learning

combines FL with transfer learning. Parties have:

- Partially overlapping users (or none)
- Different feature spaces

The shared representation is learned in a privacy-preserving manner, then transferred to enable cross-party prediction.

Example: US hospital (clinical notes) + Chinese hospital (imaging) — different patients, different languages, but shared medical concepts transferred via secure feature alignment.

Horizontal FL

Use when: Different patients, same features

Vertical FL

Use when: Same patients, different features

Transfer FL

Use when: Different patients AND features

FL Types — Comparison Table

Choose the right paradigm for your use case

Dimension	Horizontal FL	Vertical FL	Transfer FL
User overlap	None (different users)	Full (same users)	Partial / none
Feature overlap	Full (same features)	None (diff features)	None (diff features)
Privacy challenge	Gradient leakage	Entity alignment (PSI)	Feature + entity privacy
Model architecture	Identical across parties	Split model (VFL)	Transfer layers shared
Aggregation	FedAvg (weighted avg)	Secure aggregation + split	Domain-adaptive aggregation
Communication cost	Moderate (full model each rnd)	High (intermediate layers)	High (cross-domain align)
Real-world example	GBoard (Google keyboard)	Credit scoring (bank+retail)	Cross-hospital imaging
Maturity	Most mature / widely deployed	Research-stage	Early research

Cross-Silo vs. Cross-Device FL

Two fundamentally different deployment environments



Cross-Silo Federated Learning

- Small number of large participants (10–100)
- Each silo: hospital, bank, enterprise data centre
- Reliable, high-bandwidth connections
- Participants always available and trusted
- Each party has large local dataset (thousands–millions)
- Frameworks: FATE, TFF (enterprise mode)
- Governance: formal legal agreements (data sharing MoUs)



Cross-Device Federated Learning

- Millions of edge devices (smartphones, IoT)
- Each device: limited compute, battery constrained
- Unreliable, low-bandwidth mobile connections
- Participants available intermittently (sleep, charge)
- Each device has tiny local dataset (100s samples)
- Frameworks: TensorFlow Federated, PySyft
- Challenge: stragglers, dropouts, non-IID data

§3

Privacy-Preserving Techniques

Differential Privacy · Secure Aggregation · Homomorphic Encryption

Privacy Threats in Federated Learning

Gradient sharing is NOT the same as privacy — attacks still possible

Gradient Inversion

~90%

Reconstruction accuracy of training images from shared gradients (Zhu et al., 2019)

Model Inversion

High

Attacker reconstructs class representatives from model weights alone

Membership Inference

70%+

Determine whether a specific record was used in training (Shokri et al., 2017)

Data Poisoning

Subtle

Malicious client injects backdoored updates that survive aggregation

Differential Privacy — Definition

A mathematical privacy guarantee

Formal Definition (Dwork & Roth, 2014)

A randomised mechanism M is (ϵ, δ) -differentially private if for all adjacent datasets D and D' (differing by one record) and for all possible outputs S :

$$\Pr[M(D) \in S] \leq e^{\epsilon} \cdot \Pr[M(D') \in S] + \delta$$

ϵ (epsilon)

Privacy Budget

Controls how much the output can differ across adjacent datasets. Smaller ϵ = stronger privacy. $\epsilon = 0$ is perfect privacy (useless). Typical values: 0.1 – 10.

δ (delta)

Failure Probability

Probability the ϵ bound is violated. Usually set very small (10^{-5}). Allows rare "bad" events where privacy is breached.

Differential privacy ensures that the presence or absence of any single person's data has almost no effect on the published result, with ϵ controlling the strength of privacy and δ allowing a tiny probability of failure.

DP Noise Mechanisms

How randomness is calibrated to the privacy budget

Gaussian Mechanism

Formula:

$$\mathbf{M}(\mathbf{x}) = \mathbf{f}(\mathbf{x}) + \mathbf{N}(0, \sigma^2)$$

Noise scale:

$$\sigma = \Delta f \cdot \sqrt{2 \ln(1.25/\delta)} / \epsilon$$

When to use: Use for (ϵ, δ) -DP with $\delta > 0$

✓ Smooth noise, often better utility for continuous outputs

⚠ Requires $\delta > 0$ — not pure DP

Laplace Mechanism

Formula:

$$\mathbf{M}(\mathbf{x}) = \mathbf{f}(\mathbf{x}) + \text{Lap}(0, \Delta f / \epsilon)$$

Noise scale:

$$\text{Scale } b = \Delta f / \epsilon \quad (\text{sensitivity} / \text{epsilon})$$

When to use: Use for pure $(\epsilon, 0)$ -DP

✓ Stronger pure-DP guarantee

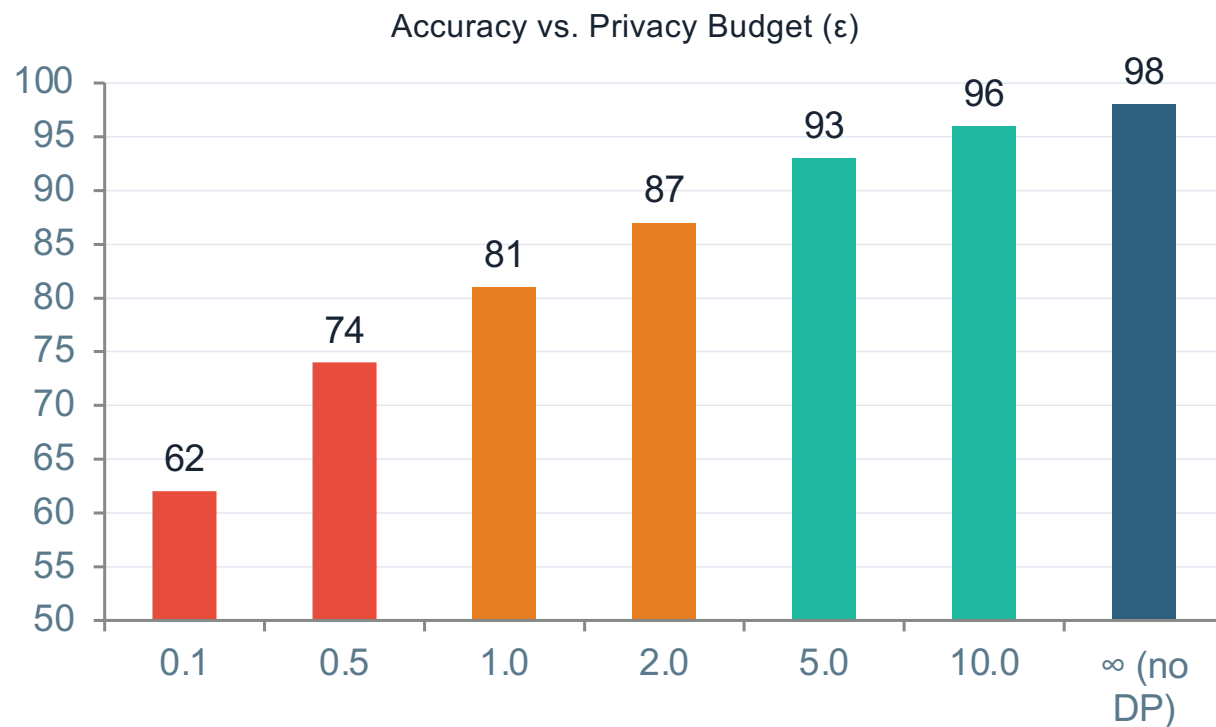
⚠ Heavier tails than Gaussian, lower utility at same ϵ

Δf (sensitivity) = max change in $f(\cdot)$ when one record is added/removed. For gradient clipping: Δf = clipping norm C .

Differential Privacy protects individuals by adding carefully calibrated random noise. Laplace noise gives stronger pure privacy guarantees $((\epsilon, 0)$ -DP), while Gaussian noise gives slightly weaker $((\epsilon, \delta)$ -DP) guarantees but is often more useful and widely used in machine learning.

The Privacy-Utility Trade-off

As ϵ decreases, privacy increases but accuracy drops



Prof. Sashikumar Ganesan | IISc Bangalore

ϵ (Privacy Budget — lower = more private)

$\epsilon \leq 1$ — Strong Privacy

Suitable for highly sensitive data (medical, financial).
Significant accuracy cost.

$1 < \epsilon \leq 5$ — Moderate Privacy

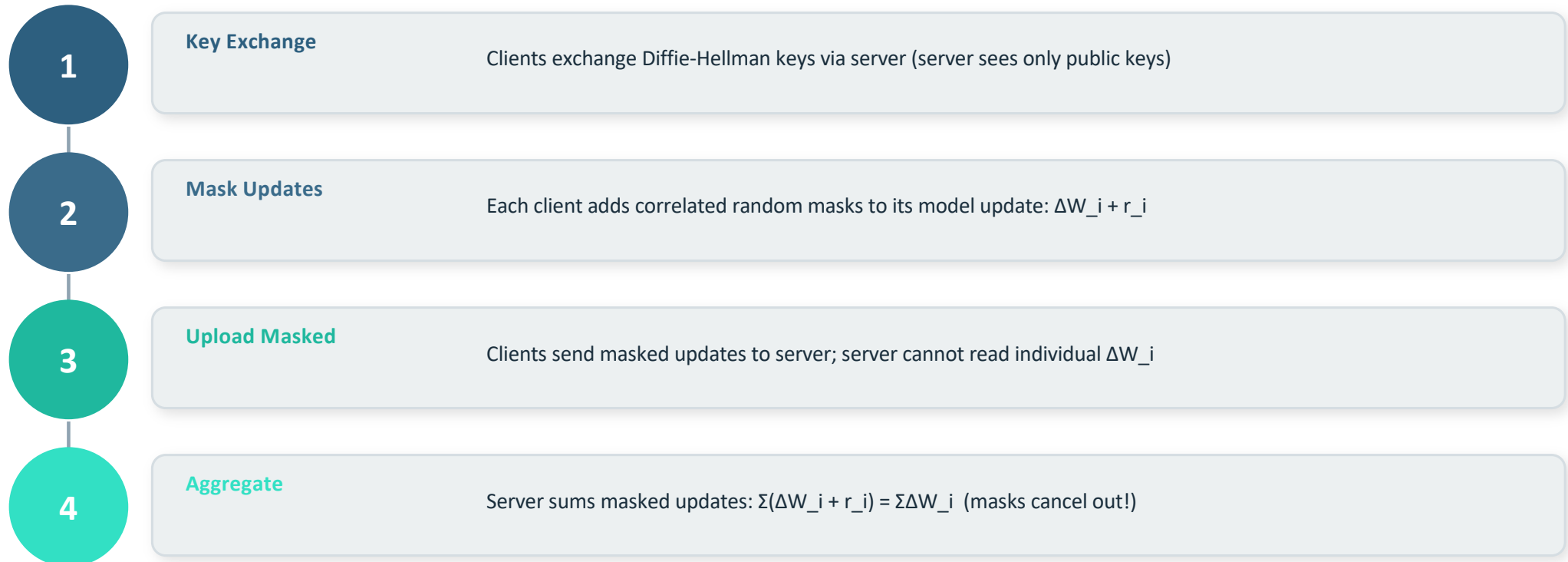
Practical for most real-world FL deployments.
Balanced trade-off.

$\epsilon > 10$ — Weak Privacy

Often used for less sensitive public/semi-public data.

Secure Aggregation (SecAgg)

Server learns only the aggregate — never individual updates



Why this works: Pairwise masks r_{ij} cancel: client i adds $+r_{ij}$, client j adds $-r_{ij}$. So $\sum r_i = 0$ and server gets true sum.

Homomorphic Encryption (HE)

Compute on encrypted data without decrypting it

What is HE?

Homomorphic Encryption allows a server to compute functions on encrypted data and return an encrypted result — the server never sees plaintext.

Key Property:

$$\text{Enc}(a) \oplus \text{Enc}(b) = \text{Enc}(a+b)$$

$$\text{Enc}(a) \otimes \text{Enc}(b) = \text{Enc}(a \times b)$$

For FL: clients encrypt their weight updates. Server aggregates in ciphertext. Only the final aggregate is decrypted.

Schemes: CKKS (approx, for ML floats), BFV/BGV (exact integer), TFHE (arbitrary circuits)

Privacy Techniques Comparison:

Technique	Privacy	Overhead
No protection	None	None
Gradient clip	Weak	Minimal
DP-SGD	Strong (tunable)	Low–Med
Sec. Aggregation	Very strong	Medium
HE (CKKS)	Strongest	Very High (100×)

Practical Reality: Full HE is 100–10,000× slower than plaintext. In FL, DP+SecAgg is the standard production combination. HE is used for specific high-security scenarios (medical, government).

The Non-IID Data Challenge

Real-world data is heterogeneous — clients have different distributions

IID (i.i.d.) Data — Ideal



Non-IID Data — Real World



Effect: Non-IID data causes client drift — local models diverge, degrading global model quality. FedAvg convergence slows; advanced methods (FedProx, SCAFFOLD) mitigate this.

FL Frameworks & Ecosystem

Choose the right tool for your deployment scenario



Flower (flwr)

Framework-agnostic, research-friendly

Best for: Custom experiments, PyTorch/TF/JAX

★★★★★ Most popular research framework



TensorFlow Federated

Google's production-grade FL stack

Best for: TF models, enterprise cross-device

★★★★☆ Production-proven



PySyft (OpenMined)

Privacy-first with HE and SecAgg support

Best for: Strong privacy guarantees, research

★★★☆☆ Research-stage



FATE

Vertical FL and enterprise cross-silo

Best for: Bank/healthcare cross-silo scenarios

★★★★☆ Enterprise-grade



FedML

Unified FL benchmark + edge deployment

Best for: Cross-device + cloud hybrid scenarios

★★★☆☆ Growing community



OpenFL (Intel)

Enterprise federated learning platform

Best for: Healthcare imaging (NVIDIA FLARE fork)

★★★☆☆ Niche enterprise

Flower (flwr) — FL in 30 Lines

Advanced Certification in Agentic & Generative AI | IISc Bangalore

Define a client, define a strategy, start the server

```
import flwr as fl
import torch, torch.nn as nn

# 1. Define FL Client
class FLClient(fl.client.NumPyClient):
    def get_parameters(self, config):
        return [val.numpy() for val in model.parameters()]
    def fit(self, params, config):
        set_params(model, params) # load global weights
        train(model, local_data) # local SGD, E epochs
        return get_params(model), len(local_data), {}
    def evaluate(self, params, config):
        loss, acc = test(model, local_data)
        return loss, len(local_data), {"accuracy": acc}

# 2. Start Server with FedAvg strategy
fl.server.start_server(
    server_address=":::8080",
    strategy=fl.server.strategy.FedAvg(
        fraction_fit=0.1, min_fit_clients=2, num_rounds=100),
)
```

Client runs on each participant's machine. Server runs centrally. Only model parameters travel over the network.

DP-SGD Implementation

Adding differential privacy to local training with Opacus

```
from opacus import PrivacyEngine
from opacus.validators import ModuleValidator

# 1. Validate + fix model for DP (no BatchNorm)
model = ModuleValidator.fix(model)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# 2. Attach PrivacyEngine — sets noise multiplier automatically
privacy_engine = PrivacyEngine()
model, optimizer, data_loader = privacy_engine.make_private_with_epsilon(
    module=model, optimizer=optimizer, data_loader=train_loader,
    epochs=10, target_epsilon=1.0, target_delta=1e-5, max_grad_norm=1.0)

# 3. Train — privacy is automatic
for batch in data_loader:
    loss = criterion(model(batch), labels); loss.backward()
    optimizer.step(); optimizer.zero_grad() # clips + noises grads
print(f"ε = {privacy_engine.get_epsilon(delta=1e-5):.2f}")
```

max_grad_norm=1.0 — Clips per-sample gradients to bound sensitivity Δf

target_epsilon=1.0 — Sets the privacy budget ϵ ; Opacus back-calculates noise σ

FL Challenges — Summary Table

Know what can go wrong before you deploy

Challenge	Root Cause	Mitigation	Severity
Non-IID data	Clients have different local distributions	FedProx, SCAFFOLD, personalisation	High
Communication cost	Many rounds × large model size	Gradient compression, sparsification	High
Stragglers	Slow/offline clients delay aggregation	Asynchronous FL, client selection	Medium
Gradient leakage	Gradients leak training data	DP-SGD, SecAgg, gradient clipping	High
System heterogeneity	Devices differ in speed/memory	Split learning, model compression	Medium
Data poisoning	Malicious client poisons updates	Robust aggregation (Byzantine FL)	High
Convergence	Non-IID + partial participation	FedAvg tuning, adaptive LR	Medium

KEY TAKEAWAYS — Sub-Module 10.1

FL is architecture, not a single algorithm Data stays local; only model parameters travel. This is a paradigm shift from centralised ML.

FedAvg is the foundation Weighted average of client weights. Know the formula: $W = \sum (n_k/n) \cdot W_k$ and what happens when clients drop out.

FL type depends on data structure Horizontal (same features), Vertical (same users), Transfer (different everything). Choose based on overlap.

Sending gradients $\neq$ privacy Gradient inversion attacks can reconstruct training data. You need DP, SecAgg, or HE on top of FL.

ϵ is a dial, not a switch Lower $\epsilon \rightarrow$ stronger privacy $\rightarrow$ lower accuracy. Set it based on data sensitivity and acceptable accuracy loss.

Non-IID data is the #1 practical challenge Real-world client data is never uniform. FedAvg degrades; consider FedProx or personalised FL.

Sub-Module 10.1 Complete

What's Next?

10.2

Distributed Training

Data parallelism · Model parallelism · Pipeline parallelism

10.3

Model Distillation

Knowledge distillation · Pruning · NAS

10.4

Applications & Challenges

Healthcare · Finance · Communication efficiency

CD-000001

Lab: Flower Implementation

Build a federated MNIST classifier with DP-SGD